PostgreSQL Index Storage Lab Report

Database: `dvdrental`
DBMS: PostgreSQL 18
Tool: pgAdmin 4

1. Objective

This lab investigates the physical storage used by PostgreSQL indexes through three tasks:

- Inspect table and index sizes.
- Measure the storage consumed by a newly created index.
- Compare a partial index with a full index.

2. Lab 1: Inspecting Table Size

Query

```
SELECT
    relname AS object_name,
    pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
    pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
FROM pg_class fed
JOIN pg_namespace n
    ON n.oid = fed.relnamespace
WHERE fed.relkind = 'r'
    AND n.nspname = 'public'
ORDER BY pg_total_relation_size(fed.oid) DESC;
```

Results

The main results shown in the output are:

Table	Data Size	Index Size	Total Size
`rental`	1232 kB	1120 kB	2352 kB
`payment`	896 kB	920 kB	1816 kB

`film`	736 kB	200 kB	936 kB
`film_actor`	272 kB	216 kB	488 kB

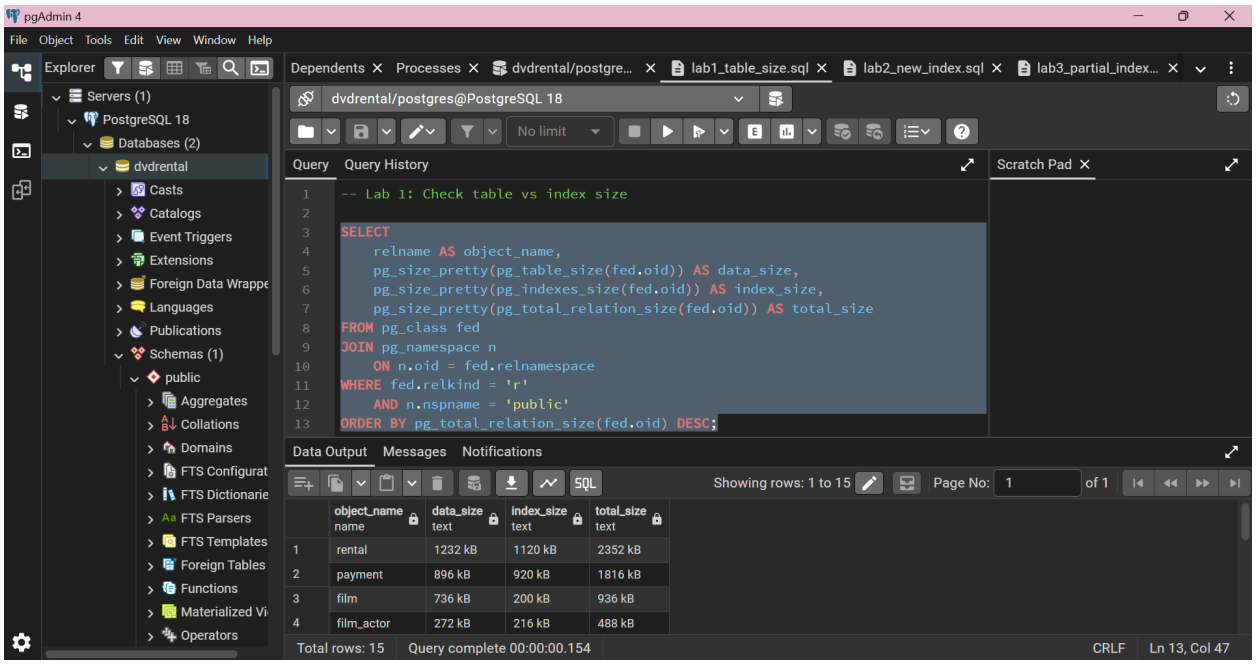

Figure 1: Lab 1 result

Observation

The `payment` table has an index size of 920 kB, which is slightly larger than its data size of 896 kB. The `rental` table also has a relatively large index size of 1120 kB compared with 1232 kB of table data.

These results show that indexes can occupy a significant amount of physical storage compared with the table data itself.

3. Lab 2: The Impact of a New Index

3.1. Objective

This lab measures the physical storage increase caused by creating a new index on the `payment` table.

3.2. Initial Index Size

The total size of the existing indexes on `payment` was checked using:

```
SELECT
    pg_size_pretty(pg_indexes_size('payment')) AS index_size_before;
```

The result was:

Index size before = 920 kB

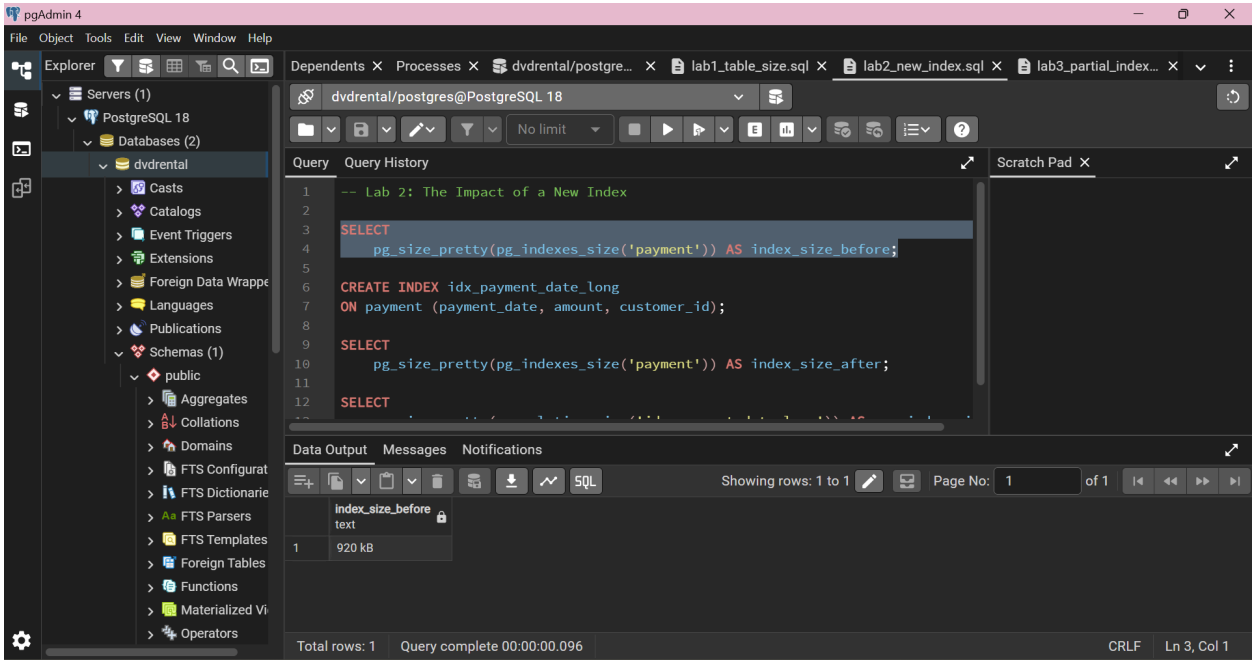

Figure 2. Total index size of the `payment` table before creating the new index.

3.3. Create the New Index

A multicolumn index was created on `payment_date`, `amount`, and `customer_id`:

```
CREATE INDEX idx_payment_date_long
ON payment (payment_date, amount, customer_id);
```

The query was executed successfully.

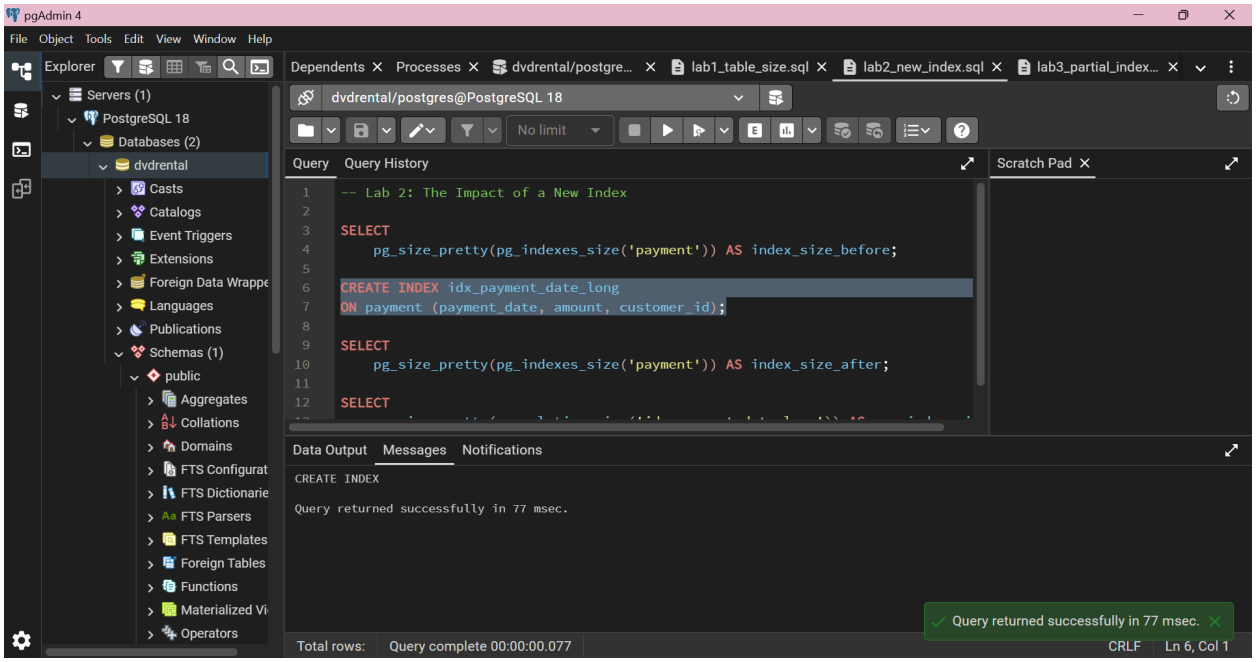

Figure 3. Successful creation of `idx_payment_date_long`.

3.4. Index Size After Creation

The total index size was checked again:

```
SELECT
    pg_size_pretty(pg_indexes_size('payment')) AS index_size_after;
```

The result was:

Index size after = 1488 kB

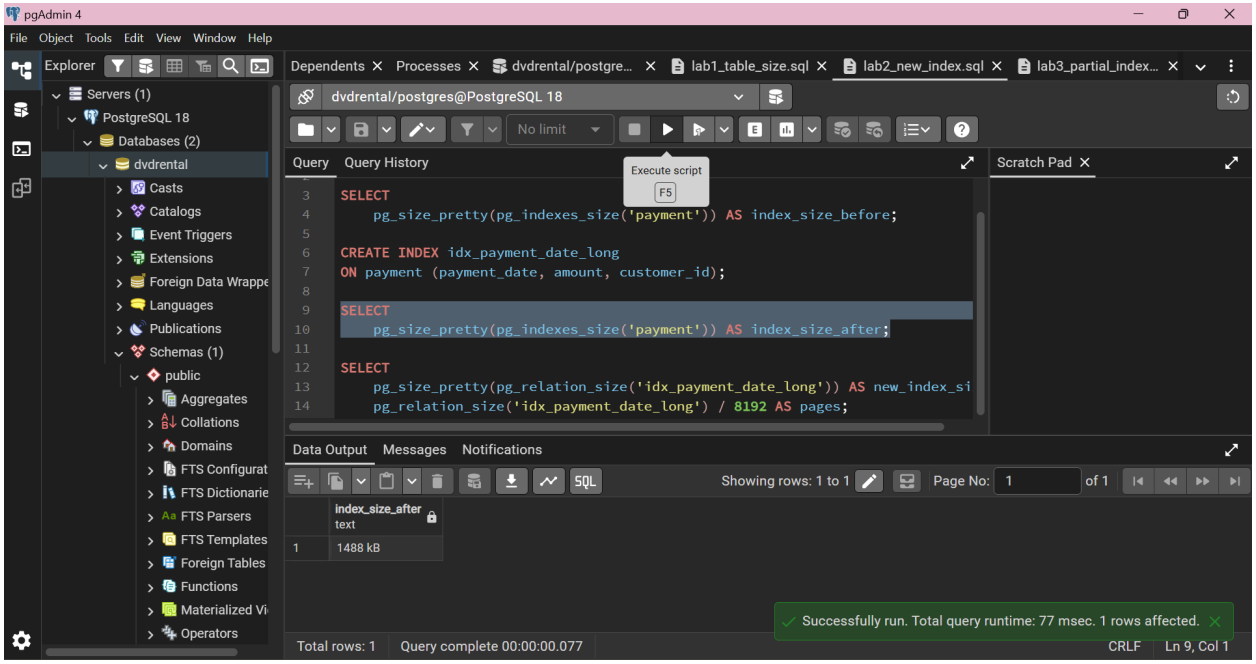

Figure 4. Total index size of the `payment` table after creating the new index.

Therefore, the storage increase was:

1488 kB - 920 kB = 568 kB

3.5. Size and Number of Pages

The size of the new index was measured directly using:

```
SELECT
    pg_size_pretty(pg_relation_size('idx_payment_date_long')) AS new_index_size,
    pg_relation_size('idx_payment_date_long') / 8192 AS pages;
```

The output shows:

- New index size: 568 kB
- Number of 8 kB pages: 71

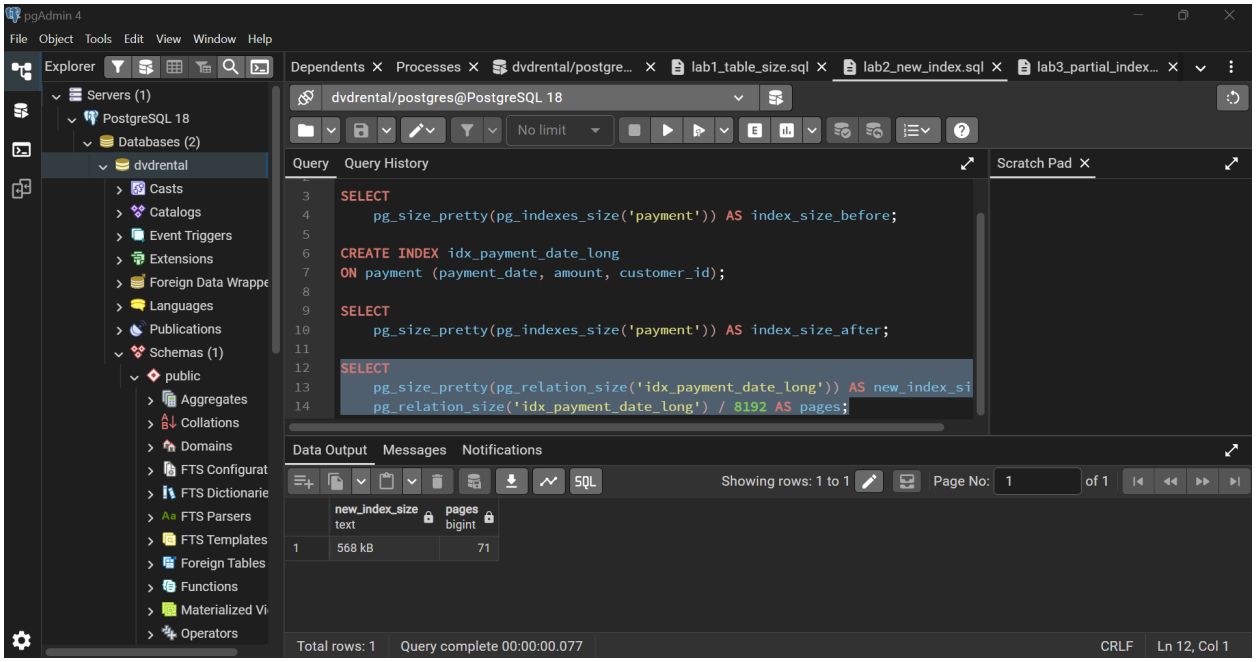

Figure 5. Physical size and number of pages used by `idx_payment_date_long`.

The measured size is consistent with the change in total index size:

1488 kB - 920 kB = 568 kB

and:

568 kB / 8 kB = 71 pages

3.6. Conclusion

Creating `idx_payment_date_long` increased the total index storage of the `payment` table from 920 kB to 1488 kB.

The new index itself occupies 568 kB, equivalent to 71 PostgreSQL pages of 8 kB each.

4. Lab 3: Partial Index Practice

4.1. Objective

This lab compares the storage required by a partial index and a full index on the `replacement_cost` column of the `film` table.

The partial index only includes films satisfying:

replacement_cost > 25.00

4.2. Create the Partial Index

The partial index was created using:

```
CREATE INDEX idx_expensive_films
ON film (replacement_cost)
WHERE replacement_cost > 25.00;
```

The query was executed successfully.

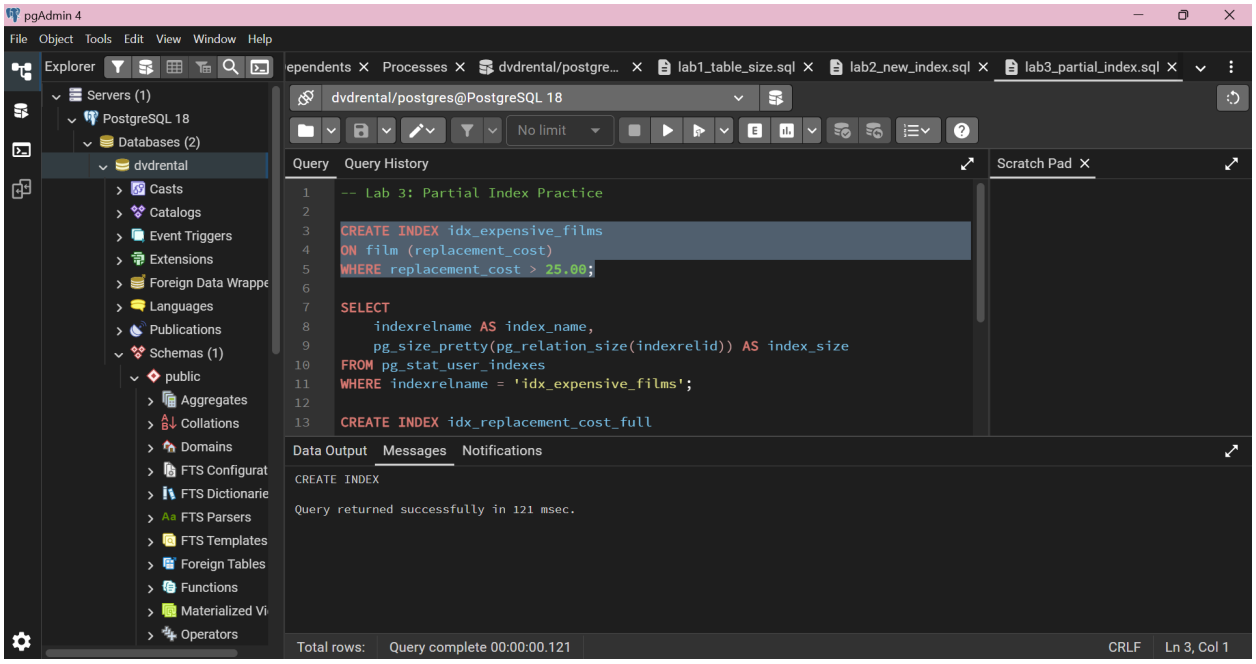

Figure 6. Successful creation of the partial index `idx_expensive_films`.

4.3. Partial Index Size

The size of the partial index was checked using:

```
SELECT
    indexrelname AS index_name,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE indexrelname = 'idx_expensive_films';
```

The result was:

Partial index size = 16 kB

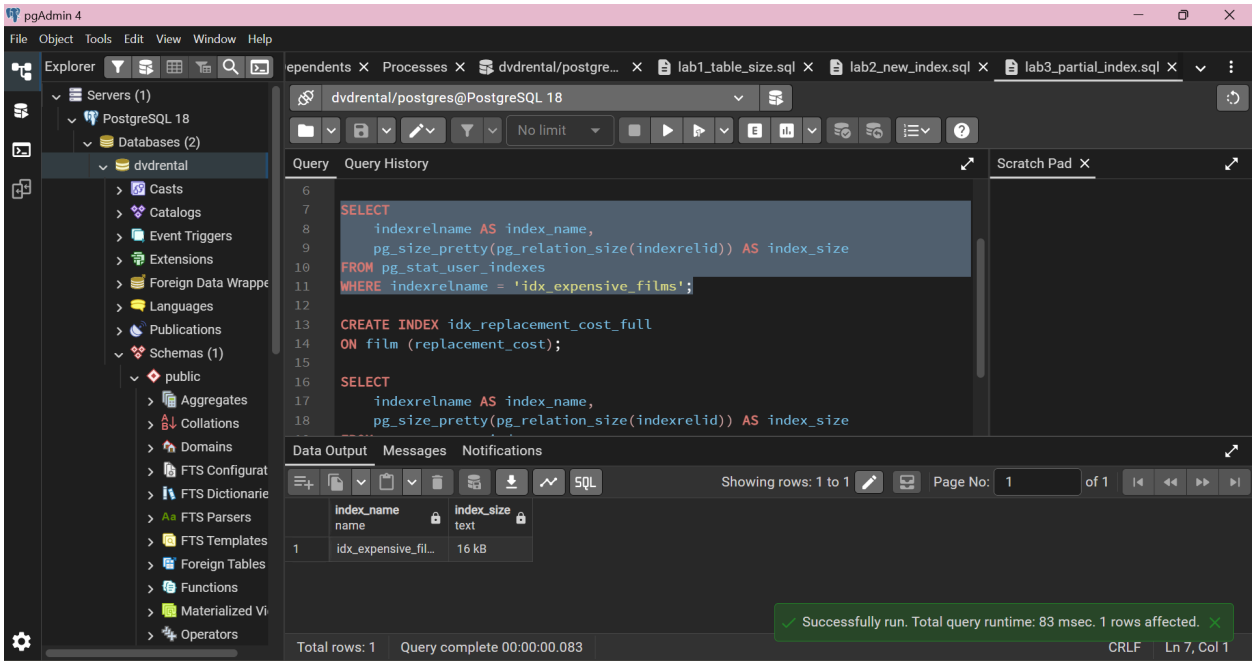

Figure 7. Physical size of the partial index.

4.4. Create the Full Index

A full index on the same column was then created for comparison:

CREATE INDEX idx_replacement_cost_full
ON film (replacement_cost);

The query was executed successfully.

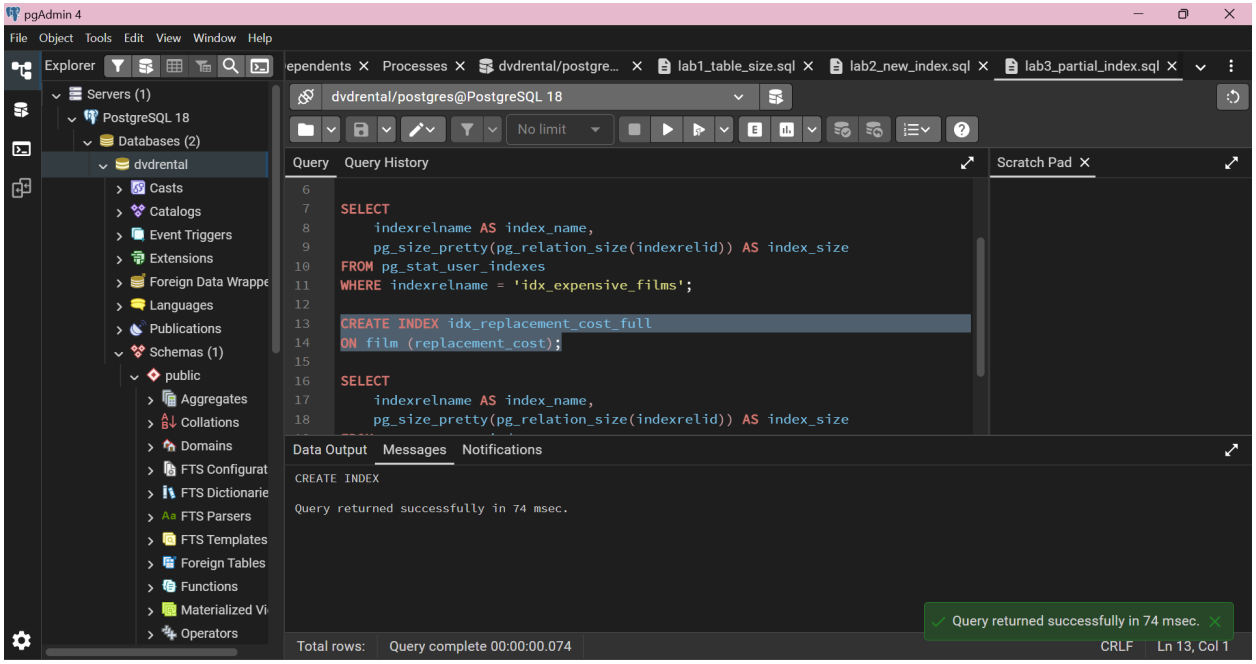

Figure 8. Successful creation of the full index on `replacement_cost`.

4.5. Compare the Two Indexes

The two index sizes were compared using:

```
SELECT
    indexrelname AS index_name,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE indexrelname IN (
    'idx_expensive_films',
    'idx_replacement_cost_full'
);
```

The output shows:

Index	Size
`idx_expensive_films`	16 kB
`idx_replacement_cost_full`	40 kB

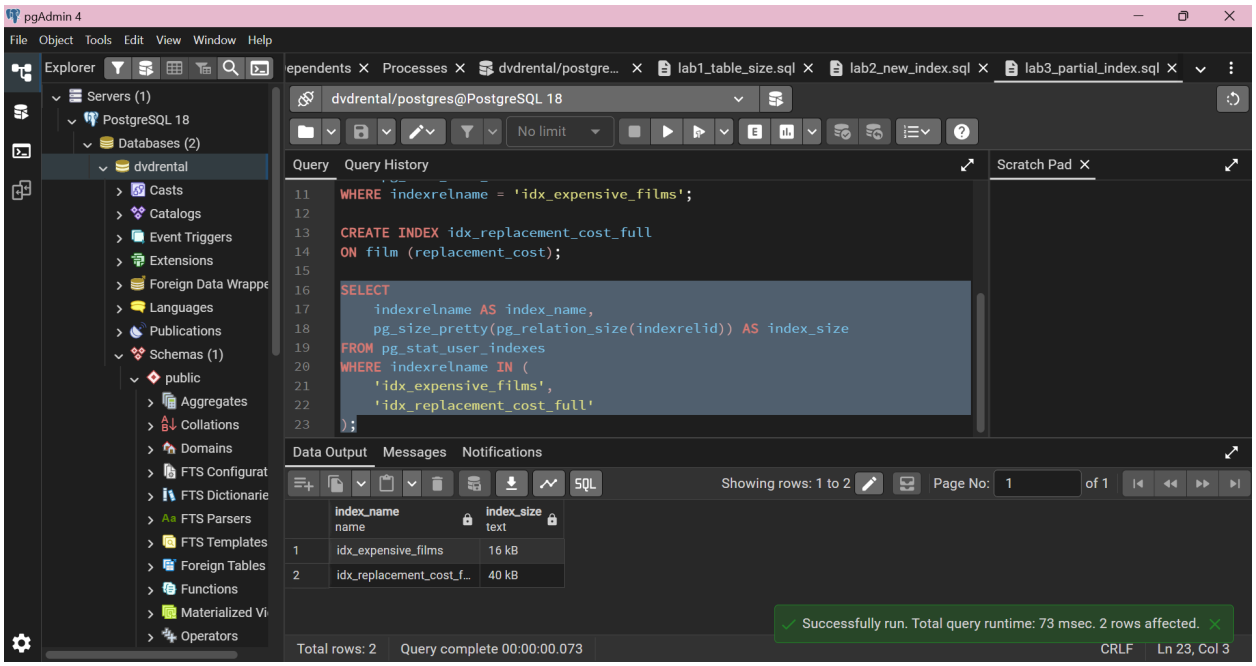

Figure 9. Comparison between the partial index and the full index.

The partial index uses:

40 kB - 16 kB = 24 kB less storage

than the full index.

This corresponds to a storage reduction of:

24 / 40 × 100% = 60%

4.6. Conclusion

The partial index occupies 16 kB, while the full index occupies 40 kB.

In this experiment, the partial index uses 24 kB less storage, corresponding to a 60% reduction compared with the full index. This occurs because the partial index only stores entries for rows where `replacement_cost > 25.00`.

5. Conclusion

The experiments demonstrate three important points about PostgreSQL indexes:

1. Indexes can occupy a significant amount of storage compared with table data.
2. Creating a new index increases the physical storage used by the database. In Lab 2, the new index occupied 568 kB or 71 pages.

3. A partial index can require less storage than a full index when only a subset of rows needs to be indexed. In Lab 3, the partial index used 16 kB, compared with 40 kB for the full index.

The SQL scripts and screenshots are available in the accompanying GitHub repository.

GitHub Repository: https://github.com/vuonghaiahn/postgresql-index-labs